

Lojistik Regresyon Temelleri

Veri Mad. – Hafta 8



Giriş ve Lojistik Regresyon Temelleri

Lojistik Regresyon (Logistic Regression), adında 'regresyon' geçmesine rağmen temel bir sınıflandırma algoritmasıdır.

Bu algoritma, Gözetimli Öğrenme (Supervised Learning) alanının en popüler doğrusal modellerinden biridir.

Temel amacı, bir örneğin belirli bir sınıfa ait olma olasılığını tahmin etmektir.



Doğrusal Regresyondan Lojistik Regresyona Geçiş

Doğrusal regresyonun çıktısı, teorik olarak $+\infty$ $-\infty$ aralığında sınırsız bir değer üretir.

Sınıflandırma problemlerinde ise ihtiyacımız olan, bir olasılık tahmini, yani $[0, 1]$ aralığında bir değerdir.

Lojistik Regresyon, bu sınırsız çıktıyı olasılık aralığına dönüştürmek için özel bir fonksiyona ihtiyaç duyar.



Sigmoid Fonksiyonunun Matematiksel Yapısı

Sınırsız doğrusal çıktıyı $[-1, 1]$ aralığına dönüştüren mekanizma, Sigmoid (Lojistik) Fonksiyon olarak adlandırılır.

Fonksiyonun denklemi şöyledir: $g(z) = 1 / (1 + e^{-z})$.

Bu S-şekilli fonksiyon, özellikle ikili (binary) sınıflandırma çıktılarını modellemek için kritik öneme sahiptir.



z Değeri: Doğrusal Kombinasyon

Sigmoid fonksiyonunun girdisi olan z , aslında doğrusal regresyondan tanıdığımız ağırlıklı özellikler toplamıdır.

z matematiksel olarak $z = \theta^T x$ şeklinde ifade edilir, burada x özellik vektörlerini, θ ise modelin öğrenmesi gereken ağırlık katsayılarını temsil eder.

Bu, girdi özelliklerinin (örneğin, Petal Length) modelin tahmini üzerindeki etkisinin bir özetidir.



Sigmoid Fonksiyonunun Grafiği ve Özellikleri

Sigmoid fonksiyonu, her zaman 0 ile 1 arasında değerler döndürür, bu da onu olasılık tahmini için ideal kılar.

Giriş z pozitif sonsuza gittikçe, çıktı 1 'e yaklaşır (birinci sınıfa ait olma olasılığı yüksek).

Giriş z negatif sonsuza gittikçe, çıktı 0 'a yaklaşır (ikinci sınıfa ait olma olasılığı yüksek).

$z=0$ noktasında, fonksiyonun değeri tam olarak 0.5 'tir, bu da karar verme sınırını oluşturur.



Karar Verme Mekanizması: Karar Sınırı (Decision Boundary)

Lojistik Regresyon modelinin karar verme süreci, Karar Sınırı (Decision Boundary) üzerine kuruludur.

Bu sınır, modelin tahmin edilen olasılığın hangi sınıfa ait olduğuna karar verdiği noktayı tanımlar.

Genellikle, tahmin edilen olasılığın 0.5'ten büyük olması bir sınıfı, küçük olması ise diğer sınıfı işaret eder.



Karar Sınırının Matematiksel Tespiti

Karar sınırı, Sigmoid fonksiyonunun tam olarak 0.5 değerini aldığı noktada çizilir.

Bu, matematiksel olarak $z=0$ olduğu duruma karşılık gelir.

$z = \theta^T x$ olduğundan, karar sınırı denklemi $\theta^T x = 0$ ile belirlenir.

Doğrusal olarak ayrılabilir verilerde bu sınır, iki boyutlu uzayda bir çizgi veya yüksek boyutlu uzayda bir düzlem (hyperplane) halini alır.



Maliyet Fonksiyonu Seçimi

Doğrusal regresyonda standart olarak kullanılan Kare Ortalama Hata (Mean Squared Error – MSE) maliyet fonksiyonu, Lojistik Regresyon için uygun değildir. Bunun nedeni, Sigmoid fonksiyonunun MSE ile birleştiğinde ortaya çıkan maliyet yüzeyinin dışbükey (non-convex) olmasıdır. Dışbükeylik, Gradyan Azalma'nın (Gradient Descent) yerel minimumlarda takılıp kalmasına yol açar ve global optimuma ulaşmasını zorlaştırır.



Olasılıksal Tahminler İçin Maliyet: Log Kaybı

Lojistik Regresyon için ideal olan maliyet fonksiyonu, Logaritmik Kayıp (Log Loss) veya İkili Çapraz Entropi (Binary Cross-Entropy) olarak adlandırılır. Bu fonksiyon, modelin olasılıksal tahminlerinin (p) gerçek etiketlere (y) ne kadar yakın olduğunu ölçer. Tahmin doğruysa maliyet düşüktür; model yüksek olasılıkla yanlış tahmin yaparsa maliyet hızla yükselir.



Log Loss (Cross-Entropy) Matematiksel İfadesi

İkili Sınıflandırma için Log Loss denklemi

Bu formül, sadece doğru sınıfa ait olan terimin maliyete katkıda bulunmasını sağlayarak matematiksel bir zarafet sunar.

Bu maliyet fonksiyonu dışbükeydir, bu sayede optimizasyon garantili bir şekilde global minimuma ulaşabilir.



Modelin Optimizasyonu: Gradyan Azalma

Modelin eğitim süreci, Log Loss maliyet fonksiyonunu minimize etmeyi amaçlar.

Bu optimizasyon, genellikle Gradyan Azalma (Gradient Descent) algoritması uygulanarak gerçekleştirilir.

Gradyan Azalma, maliyet fonksiyonunun en dik iniş yönünü bularak, model parametrelerini (ağırlıklarını) adım adım günceller.

Çok Sınıflı Lojistik Regresyon

Lojistik Regresyon, doğası gereği ikili (binary) bir sınıflandırıcı olsa da, ikiden fazla sınıfa (multinomial) sahip problemleri de çözebilir. Iris veri seti gibi üç sınıflı problemler bu duruma bir örnektir. Bu tür problemler, genellikle iki ana strateji ile ele alınır.

Strateji 1: One-vs-Rest (OvR)

One-vs-Rest (OvR) stratejisinde, N sınıflı bir problem için N adet ikili sınıflandırıcı eğitilir.

Her bir model, sadece bir sınıfı 'pozitif' (1) kabul ederken, geri kalan tüm sınıfları 'negatif' (0) olarak kabul eder.

Tahmin aşamasında, N modelin olasılık çıktıları hesaplanır ve en yüksek olasılığı veren sınıf, nihai tahmin olarak seçilir.

Scikit-learn kütüphanesi, 'LogisticRegression' sınıfında bu süreci 'multi_class='ovr' parametresi ile yönetebilir.

Strateji 2: Softmax Regresyonu

Daha teorik ve modern bir yaklaşım olan Softmax Regresyon (veya Multinomial Lojistik Regresyon), N sınıflı bir problemi tek bir modelde çözer. Softmax fonksiyonu, N farklı doğrusal fonksiyonun çıktısını alır ve bunları toplanınca 1 eden olasılık dağılımlarına dönüştürür. Bu, genellikle Softmax maliyet fonksiyonu (Çok Sınıflı Çapraz Entropi) ile optimize edilir ve OvR'a göre daha güvenilir olasılık tahminleri sunar.



Haftanın Veri Seti: Iris (Çiçek Sınıflandırması)

Iris veri seti, 1936 yılında R.A. Fisher tarafından tanıtılmıştır. Makine öğrenmesi literatüründe temel bir başlangıç noktası, yani 'Merhaba, Dünya!' veri seti olarak kabul edilir. Temiz, küçük ve iyi yapılandırılmış olması, pedagojik önemini artırır.

Iris Veri Setinin Yapısı

Veri seti toplam 150 örneklem (sample) içerir.
Bu örneklem, 4 sayısal özellik (feature) üzerinden tanımlanır.
Hedef değişken (target), 3 farklı çiçek sınıfından oluşur.



Iris Özellikleri (Features)

Lojistik regresyonun giriş değişkenleri olarak kullanılan 4 özellik şunlardır:

1. Sepal Length (Çanak Yaprak Uzunluğu)
 2. Sepal Width (Çanak Yaprak Genişliği)
 3. Petal Length (Taç Yaprak Uzunluğu)
 4. Petal Width (Taç Yaprak Genişliği)
- Bu ölçümler, santimetre cinsinden ifade edilir.

Iris Hedef Sınıfları (Target Classes)

Veri setinde sınıflandırılması gereken 3 farklı Iris türü bulunur:

1. Iris-Setosa
2. Iris-Versicolor
3. Iris-Virginica

Her sınıf, veri setinde eşit sayıda (50'şer) örneklem içerir.

Iris'in Pedagojik Önemi

Iris veri setinin Lojistik Regresyon için seçilmesinin temel nedeni, bazı özelliklerinin (özellikle Petal Length ve Petal Width) sınıfları büyük ölçüde doğrusal olarak ayırabilir kılmasıdır.

Bu özellik, Setosa sınıfının diğer iki sınıftan bir çizgi veya düzlem ile kolayca ayrılabilmesi anlamına gelir.

Bu durum, Lojistik Regresyon gibi doğrusal bir sınıflandırıcının etkinliğini net bir şekilde göstermek için idealdir.



Uygulama Ortamı ve Kütüphaneler

Pratik uygulama Google Colab ortamında gerçekleştirilecektir. Bu ortam, Python kodunu bulut tabanlı olarak çalıştırmaya olanak tanır. Kullanılacak temel kütüphaneler şunlardır:

- pandas ve numpy (Veri işleme)
- matplotlib ve seaborn (Görselleştirme)
- sklearn (Scikit-learn: Makine öğrenmesi algoritmaları ve araçları)

Verinin Yüklmesi

Iris veri seti, scikit-learn kütüphanesinin yerleşik veri setleri arasında bulunur. Veri, 'sklearn.datasets.load_iris()' fonksiyonu kullanılarak doğrudan belleğe (RAM) yüklenir.

Bu fonksiyon, veriyi genellikle numpy dizisi veya pandas DataFrame'e dönüştürülebilen bir 'Bunch' nesnesi olarak döndürür.

Veri Ayırma (Data Splitting) Kavramı

Modelin gerçek performansını objektif olarak değerlendirebilmek için verinin ayrılması zorunludur.

Modelin, eğitim verisini 'ezberlemediğinden' (overfitting) emin olmak hedeflenir.

Veri seti iki ana parçaya ayrılır:

1. Eğitim Seti (Training Set): Modelin parametreleri bu veriyi kullanarak öğrenir.
2. Test Seti (Testing Set): Modelin performansı, daha önce hiç görmediği bu veri üzerinde ölçülür.

Veri Ayırma Fonksiyonu: `train_test_split`

Veri setini ayırmak için '`sklearn.model_selection.train_test_split`' fonksiyonu kullanılır.

Bu fonksiyon, veri noktalarını rastgele karıştırarak ve genellikle 'stratify' parametresi ile sınıf oranlarını koruyarak ayırır.

Tipik olarak veri %70 eğitim ve %30 test olarak ayrılır, ancak bu oran probleme göre değişebilir.

Modelin Kurulması ve Hazırlık

Lojistik Regresyon modeli, 'sklearn.linear_model' altındaki 'LogisticRegression' sınıfı çağrılarak kurulur.

Bu sınıf çağrılırken, özellikle çok sınıflı problemler için parametrelerin belirlenmesi gerekir.



Çoklu Sınıf Parametreleri

Iris veri setinde 3 sınıf olduğu için çoklu sınıflandırma ayarları yapılmalıdır. 'multi_class='ovr' parametresi, One-vs-Rest stratejisinin kullanılacağını belirtir.

'solver='liblinear' gibi bir çözücü (optimizer) seçilir. 'liblinear', küçük veri setleri ve ikili/OvR problemler için etkili bir seçimdir.

Diğer çözücüler (saga, lbfgs), büyük veri setleri veya Softmax (multinomial) problemler için tercih edilebilir.

Modelin Eğitimi (Fitting)

Hazırlanan model, 'fit(X_train, y_train)' komutu kullanılarak eğitim verisi üzerinde eğitilir.

'X_train', eğitim veri setinin özelliklerini (features), 'y_train' ise bu özelliklere karşılık gelen gerçek hedef sınıfları (etiketleri) içerir.

Bu süreç sırasında, Gradyan Azalma, maliyet fonksiyonunu minimize edecek optimal θ hata J ağırlıklarını bulur.

Tahmin Yapma (Prediction)

Eğitim tamamlandıktan sonra, modelin performansı test edilmelidir. Model, daha önce hiç görmediği 'X_test' verisi üzerinde 'predict()' metodu kullanılarak tahmin yapmaya zorlanır. 'predict()' metodu, her bir test örneği için en yüksek olasılığa sahip sınıf etiketini döndürür. Bu tahminler, daha sonra gerçek 'y_test' değerleri ile karşılaştırılarak modelin başarısı ölçülür.



Görselleştirme: Karar Sınırları

Modelin sınıfları ayırmak için hangi bölgelerde sınır çizdiğini görsel olarak anlamak, model yorumlamanın en önemli adımlarından biridir. Bu görselleştirme, modelin doğrusal (Linear) yapısını ve hatalı sınıflandırma yaptığı alanları ortaya çıkarır. Ancak görselleştirmeyi kolaylaştırmak için, genellikle tüm özellikler yerine sadece iki özellik kullanılır.



Karar Sınırı İçin Boyut İndirgeme

Karar sınırlarının iki boyutlu bir grafikte gösterilebilmesi için, model sadece iki özellik (örneğin Petal Length ve Petal Width) kullanılarak yeniden eğitilir. Bu işlem, görselleştirme düzlemini oluşturur ve modelin bu iki özellik üzerindeki davranışını netleştirir. Bu özellikler, Iris veri setinde sınıfları ayırmada en bilgilendirici olanlardır.



Sanal Izgara (Grid) Oluřturma

Karar sınırlarını çizebilmek için, iki özelliğın oluřturduđu düzlemde yoğun, sanal bir ızgara (grid) yaratılır.

Bu ızgara, 'numpy.meshgrid' fonksiyonu kullanılarak oluřturulur.

Izgaradaki her bir nokta, potansiyel bir veri örneğini temsil eder ve bu noktalar modelin tahmin yapacağı girdi olacaktır.

Izgara Üzerinde Sınıf Tahmini

Model, oluşturulan bu sanal ızgaradaki her bir nokta için bir sınıf tahmini yapar.

Bu tahminler (etiketler), daha sonra görselleştirme için renk kodlarına dönüştürülür.

Bu, uzaydaki her noktanın, model o noktaya bir örneklem koysaydı hangi sınıfı seçeceğini gösteren bir harita oluşturur.

Sınırların Çizilmesi: `contourf` / `pcolormesh`

Izgara üzerindeki tahmin sonuçlarını görselleştirmek için '`matplotlib.pyplot.contourf()`' veya '`pcolormesh()`' fonksiyonları kullanılır. Bu fonksiyonlar, her sınıfın tahmin edildiği bölgeleri farklı renklerle boyayarak karar sınırlarını belirginleştirir.

Gerçek Veri Noktalarının Eklenmesi

Renkli tahmin bölgeleri oluşturulduktan sonra, gerçek test verisi noktaları 'plt.scatter' fonksiyonu kullanılarak grafiğin üzerine serpiştirilir. Bu adımı, modelin nerede doğru (nokta kendi rengindeki bölgede) ve nerede yanlış (nokta farklı renkteki bölgede) tahmin yaptığını görsel olarak değerlendirmemizi sağlar.



Karar Sınırı Görsel Çıktı Yorumu

Ortaya çıkan grafik, Lojistik Regresyon modelinin üç sınıf (Setosa, Versicolor, Virginica) arasında çizdiği sınırların doğrusal (düz çizgiler veya düzlemler) olduğunu net bir şekilde gösterir.

Setosa'nın diğerlerinden kolayca ayrıldığı gözlemlenirken, Versicolor ve Virginica sınıflarının sınırlarının genellikle üst üste bindiği ve karışma (overlap) olduğu görülebilir.

Bu durum, doğrusal bir modelin sınırlılıklarını ortaya koyar.



Model Performansının Değerlendirilmesi

Modelin ne kadar başarılı olduğunu objektif ve sayısal (kantitatif) olarak ölçmek kritik bir adımdır.

Bu aşamada sadece genel doğruluk (accuracy) değil, aynı zamanda hangi sınıflarda hata yapıldığı da incelenmelidir.

Değerlendirme için 'sklearn.metrics' kütüphanesinden çeşitli metrikler ve raporlama araçları kullanılır.



Temel Değerlendirme Aracı: Hata Matrisi

Hata Matrisi (Confusion Matrix), gerçek sınıflar (satırlar) ile modelin tahmin ettiği sınıflar (sütunlar) arasında bir çapraz tablo sunan güçlü bir araçtır. Çok sınıflı problemler için matris, $N \times N$ boyutunda olur (Iris için 3×3). Matrisin amacı, hangi sınıfların birbiriyle karıştırıldığını (misclassified) göstermektir.

Hata Matrisinin Yorumlanması (Köşegen)

Matrisin ana köşegeni (diagonal) üzerindeki değerler, doğru tahminleri temsil eder.

İkili sınıflandırmada bunlar: True Positives (TP) ve True Negatives (TN) olarak adlandırılır.

Çok sınıflı durumda, her sınıf için doğru tahmin edilen örnek sayısını gösterir.



Hata Matrisinin Yorumlanması (Köşegen Dışı)

Matrisin köşegen dışındaki değerleri ise modelin yaptığı hataları gösterir. Bu hatalar, False Positives (FP) ve False Negatives (FN) olarak bilinir. Örneğin, Versicolor satırında Virginica sütunundaki değer, modelin gerçekte Versicolor olan bir çiçeği yanlışlıkla Virginica olarak tahmin ettiği sayıyı gösterir .



Hata Matrisinin Görselleştirilmesi

Ham sayısal verileri daha anlaşılır kılmak için Hata Matrisi genellikle bir ısı haritası (heatmap) olarak görselleştirilir. Görselleştirme için 'seaborn.heatmap()' fonksiyonu kullanılır. 'annot=True' parametresi, hücrelerin içine ham sayısal değerlerin yazılmasını sağlayarak okumayı kolaylaştırır.

Sınıflandırma Raporu (Classification Report)

Sınıflandırma Raporu, modelin genel doğruluğunun (Accuracy) ötesine geçerek, her bir sınıf için performansı ayrıntılı olarak sunar. Bu rapor, özellikle dengesiz (imbalanced) veri setlerinde sadece doğruluğa bakmanın yetersiz olduğu durumlarda hayati öneme sahiptir. Raporda sunulan temel metrikler Precision, Recall ve F1-Score'dur.



Metrik 1: Precision (Kesinlik)

Precision (Kesinlik), modelin belirli bir sınıf için yaptığı 'pozitif' tahminlerin ne kadarının gerçekten doğru olduğunu ölçer.

Matematiksel olarak: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$

Yüksek Precision, modelin bir sınıfı tahmin ettiğinde yanılma oranının düşük olduğu anlamına gelir.



Metrik 2: Recall (Duyarlılık)

Recall (Duyarlılık), veri setindeki gerçek pozitif (ilgili) örneklerin ne kadarının model tarafından başarıyla bulunduğunu (yakalandığını) ölçer.

Matematiksel olarak: $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$

Yüksek Recall, modelin bir sınıfı kaçırma (False Negative) oranının düşük olduğu anlamına gelir.



Metrik 3: F1-Score

F1-Score, Precision ve Recall metriklerinin harmonik ortalamasıdır. Bu metrik, her iki değerin de yüksek olmasını gerektiren tek bir skor sağlar ve bu nedenle genellikle modelin dengeli performansının en iyi göstergesidir. F1-Score, özellikle Precision ve Recall arasında bir denge kurmak istendiğinde kullanılır.



Raporun Yorumlanması ve Çıkarımlar

Sınıflandırma raporu, modelin farklı sınıflardaki başarısını açıkça ortaya koyar. Örneğin, raporun Setosa sınıfı için %100 doğruluk (precision ve recall) gösterdiği, ancak Versicolor ve Virginica sınıflarında karışma nedeniyle (doğrusallık sınırı) daha düşük skorlar olduğu görülebilir. Bu yorum, modelin güçlü ve zayıf yönlerini anlamak ve iyileştirme için yol haritası çizmek için önemlidir.



Lojistik Regresyonun Avantajları

Avantajları:

1. Hız ve Basitlik: Hesaplaması hızlıdır ve büyük veri setlerinde dahi hızlıca eğitilebilir.
2. Yorumlanabilirlik: Model katsayıları (ağırlıklar), özelliklerin sınıf olasılığı üzerindeki etkisini doğrudan gösterir.
3. Olasılık Çıktısı: Sadece bir sınıf etiketi yerine, bir sınıfa ait olma olasılığını verir, bu da eşik ayarlaması (thresholding) yapma imkanı sunar.

Lojistik Regresyonun Dezavantajları

Dezavantajları:

1. Doğrusallık Varsayımı: Veri, karar sınırı ile doğrusal olarak ayrılamıyorsa (örneğin dairesel veriler), performansı önemli ölçüde düşer.
2. Outlier'lara (Aşırı Uçlara) Duyarlılık: Aykırı değerler, Sigmoid fonksiyonunun eğrisini ve dolayısıyla karar sınırını ciddi şekilde değiştirebilir.
3. Özellik Mühendisliği İhtiyacı: Karmaşık ilişkileri modelleyebilmek için genellikle ek özellik dönüşümlerine (non-linear özellikler) ihtiyaç duyar.



Eğitim ve Test Seti Kavramının Derinleştirilmesi

Eğitim setini ezberleme (Overfitting), modelin eğitim verisindeki gürültüyü veya tesadüfi desenleri öğrenmesidir.

Bu durumda model, eğitim verisinde mükemmel skorlar elde ederken, test verisinde (yani yeni veride) kötü performans sergiler.

Veri ayırma tekniği ve objektif test seti, modelin genelleme (generalization) yeteneğini ölçmek için hayati bir kontrol mekanizmasıdır.



Cross-Validation (Çapraz Doğrulama)

Tek bir train-test ayrımının ötesinde, K-Katlı Çapraz Doğrulama (K-Fold Cross-Validation) daha güvenilir bir performans tahmini sağlar.

Veri seti K eşit parçaya ayrılır.

Model K defa eğitilir; her seferinde K-1 parça eğitim için, kalan 1 parça ise test için kullanılır.

Nihai performans, bu K test sonucunun ortalaması alınarak belirlenir.



Normalizasyon ve Standardizasyonun Önemi

Gradyan Azalma (Gradient Descent) tabanlı algoritmalar, özellik ölçeklerine karşı hassastır.

Eğitimden önce özelliklerin aynı ölçeğe getirilmesi (Normalizasyon veya Standardizasyon) performansı artırır.

Bu, daha hızlı yakınsama (convergence) sağlar ve modelin katsayılarının aşırı değerler almasını engeller.



Regülerizasyon (Düzenleştirme) Kavramı

Regülerizasyon, modelin aşırı karmaşıklaşmasını ve katsayıların (ağırlıkların) çok büyük değerler almasını engelleyen bir tekniktir.

Lojistik Regresyon genellikle L1 (Lasso) veya L2 (Ridge) regülerizasyonları ile kullanılır.

'LogisticRegression' sınıfında, 'penalty' parametresi ile regülerizasyon tipi belirlenir ve 'C' (ters regülerizasyon gücü) ayarlanır.



ROC Eğrisi ve AUC Metriği

İkili sınıflandırmada, ROC Eğrisi (Receiver Operating Characteristic Curve) ve Eğri Altındaki Alan (AUC - Area Under the Curve) popüler metriklerdir. ROC eğrisi, modelin tüm olası karar eşiklerinde (thresholds) True Positive Rate (Duyarlılık) ve False Positive Rate arasındaki dengeyi gösterir. AUC değeri, modelin sınıfları ne kadar iyi ayırabildiğinin tek bir sayısal özetidir (1.0 mükemmel ayırma demektir).



Kullanılan Temel Kütüphanelerin Detayları

Numpy: Hızlı ve verimli matris (array) işlemleri için temeldir. 'meshgrid' gibi yapıları sağlar.

Pandas: Veri manipülasyonu ve analiz için kullanılır. Veri setlerini DataFrame yapısında tutmaya yarar.

Matplotlib/Seaborn: Statik, interaktif ve profesyonel kalitede görselleştirmeler oluşturur. Karar sınırları ve heatmap'ler için kritiktir.

Scikit-learn (sklearn): Model eğitimi, veri ayırma ('train_test_split') ve metrik hesaplama ('metrics') gibi tüm makine öğrenmesi iş akışını yönetir.

Kodlama Adım 1: Kütüphane İçe Aktarımı

Python uygulamasının ilk adımı, gerekli tüm modülleri ve sınıfları içe aktarmaktır.

- 'import numpy as np'
- 'import matplotlib.pyplot as plt'
- 'from sklearn.datasets import load_iris'
- 'from sklearn.model_selection import train_test_split'
- 'from sklearn.linear_model import LogisticRegression'
- 'from sklearn.metrics import classification_report, confusion_matrix'



Kodlama Adım 2: Veri Yükleme ve Yapılandırma

'iris = load_iris()' komutu ile veri yüklenir.
Özellikler (features) 'X = iris.data' olarak, hedef değişken (target) ise 'y = iris.target' olarak atanır.
Eğitim için verinin sayısal matrisler halinde hazır olması sağlanır.

Kodlama Adım 3: Veri Ayırma Uygulaması

```
"""python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)"""
```

'test_size=0.3' test setinin %30 olacağını belirtir.

'random_state' rastgeleliği sabitleyerek sonuçların tekrarlanabilirliğini sağlar.

'stratify=y' ise sınıfların oranlarının hem eğitim hem de test setinde korunmasını garanti eder.



Kodlama Adım 4: Modelin Tanımlanması ve Eğitimi (Tüm Özellikler)

```
"""python
model = LogisticRegression(
    multi_class='ovr', solver='liblinear', random_state=42, max_iter=200
)"""
```

'max_iter' parametresi, modelin yakınsayana kadar deneyeceği maksimum iterasyon sayısını belirler.

'model.fit(X_train, y_train)' komutu ile model eğitilir.

Kodlama Adım 5: Tahmin ve Performans Hesaplama

Tahminler test verisi üzerinden yapılır:

```
'y_pred = model.predict(X_test)'
```

Modelin performansını ölçmek için metrikler hesaplanır:

```
'print(classification_report(y_test, y_pred))'
```

```
'cm = confusion_matrix(y_test, y_pred)'
```

Kodlama Adım 6: Karar Sınırları İçin Veri Hazırlığı

Görselleştirme için, sadece Taç Yaprak Uzunluğu (Petal Length, index 2) ve Taç Yaprak Genişliği (Petal Width, index 3) kullanılır.

```
python
X_2d = X[:,]
y_2d = y
X_train_2d, X_test_2d, y_train_2d, y_test_2d = train_test_split(
    X_2d, y_2d, test_size=0.3, random_state=42, stratify=y_2d
)
```

Kodlama Adım 7: 2D Model Eğitimi ve Meshgrid Oluşturma

Sadece 2 özelliği kullanan yeni bir Lojistik Regresyon modeli eğitilir.

```
'model_2d = LogisticRegression(...).'
```

```
'model_2d.fit(X_train_2d, y_train_2d).'
```

Özellik aralıkları belirlenir ve 'np.meshgrid' kullanılarak ızgara noktaları oluşturulur.

Bu ızgara noktaları daha sonra model ile tahmin edilmek üzere düzleştirilir.

Kodlama Adım 8: Karar Sınırlarının Çizimi

Modelin ızgara üzerindeki tahminleri renklendirilmiş bölgeler olarak çizilir:

```
python
```

```
Z = Z.reshape(xx.shape)
```

```
plt.contourf(xx, yy, Z, alpha=0.3, cmap=cmap)"""
```

'contourf' fonksiyonu, renkli dolgu alanlarını oluşturur ve 'alpha' ile şeffaflık ayarlanır .

Ardından gerçek test noktaları eklenir:

```
'plt.scatter(X_test_2d[:, 0], X_test_2d[:, 1], c=y_test_2d, ...)'
```

Karar Sınırı Örneği: Setosa'nın Ayrılması

Iris-Setosa sınıfının, Taç Yaprak (Petal) ölçüleri grafiğinde diğer sınıflardan fiziksel olarak çok uzakta kümelendiği görülür. Lojistik Regresyon, bu ayrılabilirliği kullanarak Setosa'yı neredeyse %100 doğrulukla ayıran net bir doğrusal sınır çizer. Bu durum, modelin basit ama etkili olduğunu gösterir.



Karar Sınırı Örneği: Versicolor ve Virginica Karışımı

Iris-Versicolor ve Iris-Virginica sınıfları, özellik uzayında (özellikle petal ölçülerinde) önemli ölçüde üst üste biner (overlap). Bu durum, sınıfların tam olarak doğrusal olarak ayıramadığı anlamına gelir. Model, bu iki sınıf arasında yine düz bir çizgi çizer, ancak bu çizgi, karışma bölgesindeki birçok noktayı yanlış sınıflandırmaya (hata matrisinde FN/FP'ye) neden olur.



Bias-Variance Takası (Dengelemesi)

Lojistik Regresyon, yüksek yanlılığa (High Bias) sahip bir modeldir, yani varsayımları basittir (doğrusallık). Bu, genelleme yeteneği iyidir ancak karmaşık veriyi öğrenemez.

Bias (Yanlılık): Modelin eğitim verisinden beklenen doğru sonuç ile gerçek sonuç arasındaki fark.

Variance (Varyans): Modelin, eğitim verisindeki küçük değişikliklere ne kadar duyarlı olduğu (yüksek varyans = overfitting riski).

Multinomial Log Loss (Çok Sınıflı Çapraz Entropi)

Softmax Regresyonu kullanılırken, Binary Cross-Entropy yerine Multinomial Log Loss kullanılır.

Bu maliyet fonksiyonu, N farklı sınıf olasılığının tahminlerini aynı anda değerlendirir ve tahmin edilen olasılık dağılımı ile gerçek olasılık dağılımı arasındaki mesafeyi ölçer.

Eğitim, bu maliyet fonksiyonunu minimize ederek, her bir sınıfa ait olma olasılığının doğru bir şekilde hesaplanmasını hedefler.



Model Çıktısı: Tahmin Olasılıkları

Model sadece 'predict()' ile nihai sınıf etiketini değil, aynı zamanda 'predict_proba()' metodu ile her bir sınıfa ait olma olasılıklarını da döndürür. Bu olasılıklar, karar eşiğini (örneğin 0.5) farklı değerlere ayarlamak veya belirsizlik ölçümü yapmak için kullanılabilir. Üç sınıflı Iris örneğinde, 'predict_proba' her örnek için 3 olasılık değeri (toplamları 1) döndürür.

Lojistik Regresyon ve Uygulama Alanları

Lojistik Regresyon, basitliği ve yorumlanabilirliği sayesinde endüstride hala yaygın olarak kullanılmaktadır:

1. Tıbbi Teşhis: Bir hastanın belirli bir hastalığa yakalanma olasılığının tahmini (Binary)
2. Finans: Kredi risk analizi (bir müşterinin temerrüde düşüp düşmeyeceği)
3. Pazarlama: Bir müşterinin bir ürünü satın alma olasılığının tahmini (Churn Prediction).



Hata Matrisi Görselleştirme Kodu

```
"""python
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.ylabel('Gerçek Sınıf')
plt.xlabel('Tahmin Edilen Sınıf')
plt.title('Hata Matrisi')"""
'annot=True' ham sayıların grafikte görünmesini sağlar. 'fmt='d' formatının
tam sayı olduğunu belirtir.
```

Sınıflandırma Raporunun İncelemesi

Çıktı Raporu Örneği:

- Setosa: Precision 1.00, Recall 1.00 (Mükemmel ayrılabilirlik)
- Versicolor: Precision 0.95, Recall 0.90 (Bazı karışmalar)
- Virginica: Precision 0.90, Recall 0.95 (Bazı karışmalar)

Bu rapor, modelin Setosa sınıfını %100 doğrulukla tanırken, Versicolor ve Virginica sınıflarını belirli bir oranda karıştırdığını açıkça ortaya koyar.

Genel Doğruluk (Accuracy) Metriği

Doğruluk (Accuracy), tüm doğru tahminlerin toplamının, toplam örneklem sayısına bölünmesiyle elde edilen yüzdendir.

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Bu, modelin ne kadarının genel olarak doğru tahmin edildiğini gösterir.

Dengesiz veri setlerinde (imbalanced data), tek başına doğruluk metriği yanıltıcı olabilir ve Precision/Recall'ün incelenmesi zorunludur.



Makro, Mikro ve Ağırlıklı Ortalama F1-Score

Çok sınıflı raporlarda, tüm sınıfların Precision ve Recall değerlerini tek bir skorda özetlemek için ortalama alma yöntemleri kullanılır:

1. Mikro Ortalama: Her bir örneklem düzeyinde toplam TP/FP/FN hesaplanır.
2. Makro Ortalama: Her sınıf için metrikler hesaplanır ve basit aritmetik ortalaması alınır.
3. Ağırlıklı Ortalama: Her sınıfın metriği, o sınıftaki örneklem sayısıyla (support) ağırlıklandırılarak hesaplanır. (Dengesiz veriler için daha iyidir)

Lojistik Regresyonun Ardındaki Olasılıksal Çerçeve

Lojistik Regresyon, parametrelerini (ağırlıklarını) aslında Maksimum Olabilirlik Tahmini (Maximum Likelihood Estimation - MLE) prensibine göre bulur. MLE, gözlemlenen veri setinin ortaya çıkma olasılığını en üst düzeye çıkaracak model parametrelerini arar. Log Loss maliyet fonksiyonunu minimize etmek, matematiksel olarak MLE'yi maksimize etmeye eşdeğerdir; bu, modelin sağlam istatistiksel temellere dayandığını gösterir.



Sonuç: Lojistik Regresyonun Önemi

Lojistik Regresyon, makine öğrenmesi yolculuğunun kritik bir başlangıcıdır. Sigmoid, Log Loss, Karar Sınırları ve Gradyan Azalma gibi temel kavramları somutlaştırır.

Iris veri seti üzerinde çalışmak, modelin doğrusal ayrılabilirlik varsayımını ve performans metriklerinin yorumlanmasını uygulamalı olarak gösterir.



Lojistik Regresyon Temelleri

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

